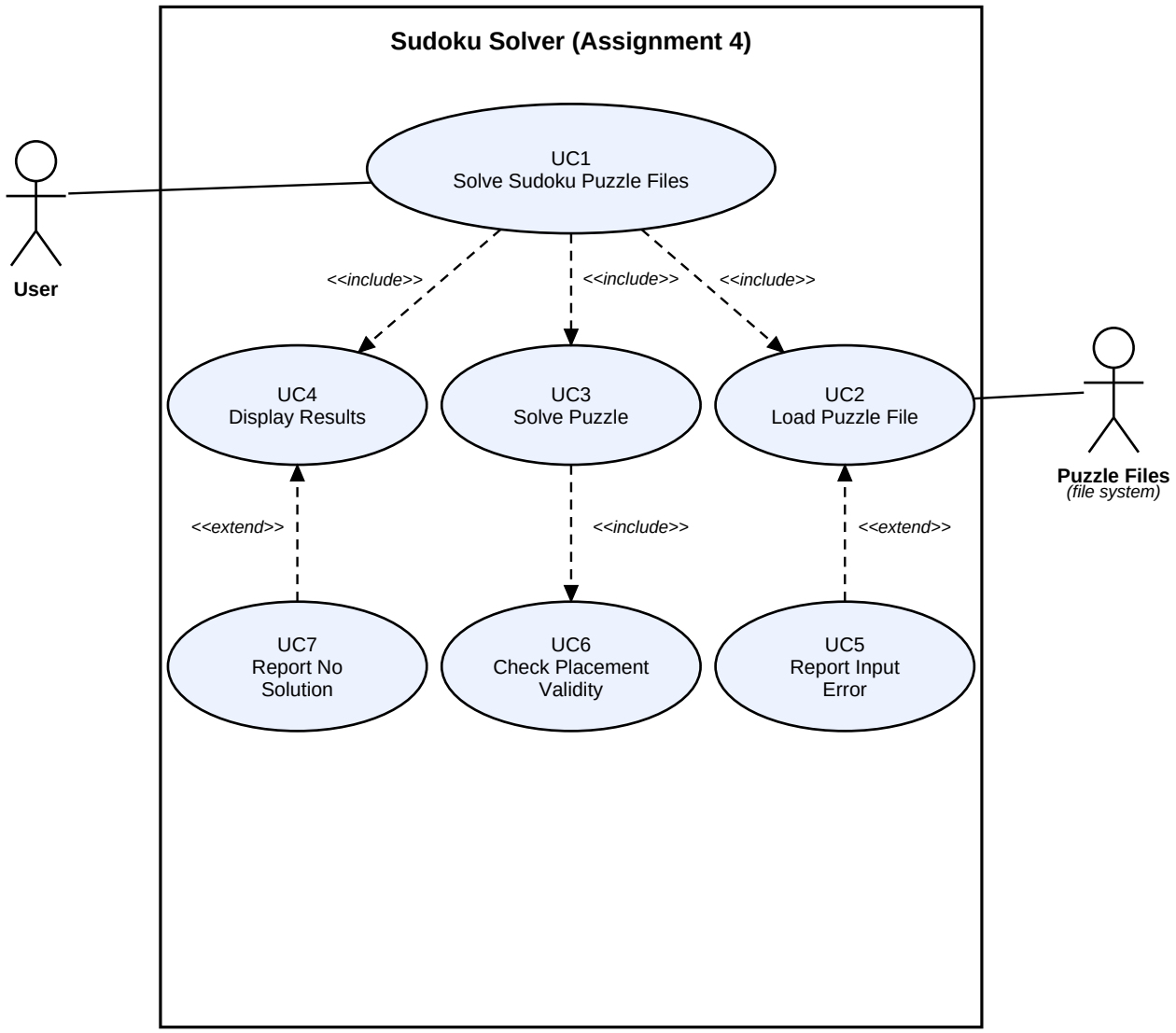
EECS 348 Assignment 4: UML Use Case Diagram

Sudoku solver (depth-first search with backtracking). Author: Lucas Frias. Diagram and descriptions drafted with a GenAI assistant, then matched to the program's behavior.

1. Use case diagram



Primary actor: **User** (starts the program from a terminal). Secondary actor: **Puzzle Files** (puzzle1.txt to puzzle5.txt, read by the program). UC1 includes UC2, UC3 and UC4. UC3 includes UC6. UC5 extends UC2 and UC7 extends UC4.

2. Identified use cases

ID	Use case	Relationship	Brief description
UC1	Solve Sudoku Puzzle Files	Primary; includes UC2, UC3, UC4	User runs the program and every puzzle file is loaded, solved and reported.
UC2	Load Puzzle File	Included by UC1	Read 81 cells from one file into the 9x9 grid.
UC3	Solve Puzzle	Included by UC1; includes UC6	Search for every valid completion by recursion and backtracking.
UC4	Display Results	Included by UC1	Print file name, starting puzzle, each solution and the solution count.
UC5	Report Input Error	Extends UC2	Print an error and skip the file when it cannot be read or parsed.
UC6	Check Placement Validity	Included by UC3	Test whether a digit fits a cell under row, column and block rules.
UC7	Report No Solution	Extends UC4	Print "No solution found" when the search finds zero solutions.

3. Use case descriptions

Use case	UC1: Solve Sudoku Puzzle Files
Actors	User (primary); Puzzle Files (secondary)
Brief description	The user runs the program. It processes puzzle1.txt through puzzle5.txt in order and prints every solution for each.
Preconditions	Program is compiled and run from a folder that holds puzzle1.txt to puzzle5.txt.
Step-by-step flow	1. User runs the executable with no arguments. 2. System builds the list of five puzzle file names. 3. For each file, system creates a solver object for that file name. 4. System loads the puzzle (UC2). If loading fails, skip to the next file. 5. System prints the file name and starting puzzle, then solves it (UC3). 6. System displays the results (UC4). 7. After the fifth file, program exits with code 0.
Alternate flows	A file that fails to load does not stop the run. The remaining files are still processed.
Postconditions	Every readable puzzle file has been solved and reported. Program has exited.
Detailed description	This is the top-level goal. The main routine holds the five file names and loops over them. Each pass creates one solver object, so no state carries over between puzzles. The user sees all output in the terminal. The run is fully batch: there is no prompt and no input after start.

Use case	UC2: Load Puzzle File
Actors	Puzzle Files (secondary)
Brief description	Read one puzzle file and fill the 9x9 grid. An underscore becomes an empty cell. A digit 1 to 9 becomes a given.
Preconditions	Solver object exists with an empty grid and a file name.
Step-by-step flow	1. Open the file. 2. For row 0 to 8 and column 0 to 8, read the next non-whitespace character. 3. If the character is '_', store 0 (empty). 4. If the character is '1' to '9', store that digit. 5. After 81 cells are stored, return success.
Alternate flows	File cannot be opened, fewer than 81 cells exist, or a character is not '_' or 1-9: run UC5 and return failure.
Postconditions	Grid holds the puzzle and loading returned true, or an error was printed and loading returned false.
Detailed description	Whitespace and newlines are skipped, so spacing in the file does not matter. Cells are read left to right, top to bottom. Characters after the 81st cell are not checked, so a file with extra digits on its last row is accepted and the excess is ignored. Givens are not checked against each other at load time.

Use case	UC3: Solve Puzzle
Actors	None (internal)
Brief description	Find every solution by brute force. Try each digit in each empty cell, recurse, and undo the try on return.
Preconditions	Grid is loaded. Solution count is zero.
Step-by-step flow	1. Start at row 0, column 0. 2. If the row index passes row 8, a full grid exists: count it and print it (UC4), then return. 3. If the current cell holds a digit, recurse on the next cell. 4. If the cell is empty, try digits 1 to 9 in order. 5. For each digit, check validity (UC6). If valid, place it and recurse on the next cell. 6. When the recursive call returns, reset the cell to empty (backtrack) and try the next digit. 7. When all digits are tried, return to the previous cell.
Alternate flows	A cell where no digit passes UC6 ends that branch. The search backs up and tries something else.
Postconditions	All branches are explored. Grid is back to its starting state. Solution count is final.
Detailed description	This is depth-first search with backtracking. Worst-case time is about 9^k for k empty cells. Because it never stops at the first answer, it finds all solutions, as the assignment requires. Memory is the grid plus one stack frame per cell, at most 81 deep. Each solution is printed the moment it is found and not stored.

Use case	UC4: Display Results
Actors	User (reads output)
Brief description	Print the file name, the starting puzzle, every solution, and a final count.
Preconditions	Solver has started on a loaded puzzle.
Step-by-step flow	1. Print a separator line and "File: <name>". 2. Print the starting puzzle with '_' for empty cells. 3. As UC3 finds each solution, print "Solution N:" and the grid. 4. When the search ends, print "Solutions found: N". 5. Print a closing separator line.
Alternate flows	If N is zero, run UC7 instead of printing the count.
Postconditions	Terminal shows the complete report for that puzzle.
Detailed description	Output is plain text, one grid per block, nine rows of nine cells separated by spaces. Multiple solutions appear as Solution 1, Solution 2 and so on, so non-unique puzzles are fully listed.

Use case	UC5: Report Input Error
Actors	User (reads output)
Brief description	Print a clear error for a bad file and skip it.
Preconditions	UC2 hit a problem.
Step-by-step flow	1. Identify the problem: cannot open, too few cells, or bad character. 2. Print "Error: <reason>" with the file name (and the bad character if any) to the error stream. 3. Return failure to the caller.
Alternate flows	None.
Postconditions	No solve is attempted for this file. The next file still runs.
Detailed description	Three messages exist: could not open file, invalid or incomplete puzzle format, and invalid character. Errors go to standard error so they stay separate from normal output.

Use case	UC6: Check Placement Validity
Actors	None (internal)
Brief description	Decide whether a digit may be placed in a given cell.
Preconditions	A row, column and digit are given.
Step-by-step flow	1. Scan the nine cells of the row and the nine of the column for the digit. 2. If found in either, return false. 3. Compute the top-left corner of the 3x3 block that holds the cell. 4. Scan the nine cells of the block for the digit. 5. If found, return false. Otherwise return true.
Alternate flows	None.
Postconditions	Grid is unchanged. A boolean is returned.
Detailed description	This is the constraint rule of Sudoku: a digit appears once per row, once per column, and once per block. It runs on every try, so it is the hot path of the program. Cost is constant per call.

Use case	UC7: Report No Solution
Actors	User (reads output)
Brief description	Tell the user that no completion exists for a puzzle.
Preconditions	UC3 finished with a solution count of zero.
Step-by-step flow	1. Check that the count is zero. 2. Print "No solution found". 3. Print the closing separator line.
Alternate flows	None.
Postconditions	Report for that file is complete.
Detailed description	A puzzle has no solution when its givens force a contradiction. The search proves this by exhausting every branch. Example: puzzle4.txt in the test set.